

QUALIFICAÇÃO PROFISSIONAL — PROGRAMADOR FRONT-END

Desenvolvimento Web

Guia de Estudos — Rotas Parte 2: Parâmetros e Navegação Programática

Continuação do projeto `Routes` criado na Aula 1 (com as páginas Início, Sobre, Contato e Ajuda). Aqui você vai aprender a criar uma página de detalhe reutilizável para cada item de uma lista, navegar por código em vez de clicar num link, e tratar endereços que não existem.

`useParams` • `useNavigate` • Rota Coringa (404)

Sumário

1. **Recapitulando a Aula 1** — o que já temos no projeto Routes
2. **Rotas com Parâmetro** — uma página para representar vários itens
3. `useParams` — lendo o parâmetro e buscando o item certo
4. **Navegação Programática** — `useNavigate`
5. **Página 404** — a rota coringa

1. Recapitulando a Aula 1

O que já existe no projeto antes de começar esta aula.

Na Aula 1, montamos o projeto `Routes` com quatro páginas fixas — cada uma com um caminho próprio, sempre igual:

Arquivo	Rota
<code>src/pages/Home.jsx</code>	<code>/</code>
<code>src/pages/Sobre.jsx</code>	<code>/sobre</code>
<code>src/pages/Contato.jsx</code>	<code>/contato</code>
<code>src/pages/Ajuda.jsx</code>	<code>/ajuda</code>

Isso funciona bem para páginas que são sempre as mesmas. Mas hoje o desafio é diferente: como fazer uma página de detalhe para **cada produto de uma lista**, sem escrever uma rota (e um arquivo) para cada um deles? É isso que vamos resolver agora, criando duas páginas novas: `Produtos` (a lista) e `ProdutoDetalhe` (o detalhe de um item específico, reaproveitado para todos).

2. Rotas com Parâmetro

Uma única rota para representar vários itens diferentes.

A solução é uma rota com **parâmetro dinâmico**, marcado com `:` no caminho:

```
<Route path="/produtos/:id" element={<ProdutoDetalhe />} />
```

Tanto `/produtos/1` quanto `/produtos/42` batem com essa mesma rota — o valor depois de `/produtos/` fica disponível dentro do componente. É uma única página, capaz de se comportar de forma diferente dependendo do endereço.

Analogia

Pense na rota `/produtos/:id` como um formulário de rifa com um campo em branco: "Produto número ____". O `:id` é o espaço em branco, e cada acesso diferente (`/produtos/1`, `/produtos/2`) preenche esse espaço com um valor diferente.

A página da lista

src/pages/Produtos.jsx

```
import { Link } from "react-router-dom";

const produtos = [
  { id: "1", nome: "Teclado Mecânico", descricao: "Switches lineares e RGB per-key." },
  { id: "2", nome: "Mouse Gamer", descricao: "Sensor de alta precisão." },
  { id: "3", nome: "Headset Gamer", descricao: "Áudio imersivo 7.1." },
];

export default function Produtos() {
  return (
    <div>
      <h1>Produtos</h1>
      <ul>
        {produtos.map((produto) => (
          <li key={produto.id}>
            <Link to={`/${produtos}/${produto.id}`}>{produto.nome}</Link>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

Explicando bloco a bloco

- `const produtos = [...]` — um array comum de objetos, direto no arquivo (mais adiante no curso isso vai vir de uma API; por enquanto, fica assim, escrito à mão).
- `{produtos.map((produto) => (...))}` — para cada produto do array, cria um `<li>` com um link. O mesmo padrão de lista que já usamos ao renderizar vários componentes a partir de um array.
- `<Link to={`/produtos/${produto.id}`}>` — monta o endereço **dinamicamente**, usando template string (crase) para juntar o texto fixo `/produtos/` com o `id` de cada produto. Para o primeiro produto, isso vira `/produtos/1`.
- `key={produto.id}` — obrigatório sempre que se usa `.map()` para gerar uma lista de elementos.

3. `useParams` — Lendo o Parâmetro

Descobrimos, dentro do componente, qual item foi acessado.

```
import { useParams } from "react-router-dom";

function ProdutoDetalhe() {
  const { id } = useParams();
  return <h1>Produto {id}</h1>;
}
```

`useParams()` devolve um objeto com todos os parâmetros da rota atual. Como a rota se chama `:id`, a chave no objeto também se chama `id`. Se o endereço for `/produtos/2`, o `id` dentro do componente será a string `"2"`.

Atenção

`useParams()` sempre devolve **strings** — mesmo que pareça um número. Comparar `id === 2` (número) nunca vai bater; o certo é comparar com `id === "2"` (string), exatamente como fizemos no array de produtos, onde os `id` também foram escritos entre aspas.

A página de detalhe completa

```
src/pages/ProdutoDetalhe.jsx
```

```
import { useParams, useNavigate } from "react-router-dom";

const produtos = [
  { id: "1", nome: "Teclado Mecânico", descricao: "Switches lineares e RGB per-key." },
  { id: "2", nome: "Mouse Gamer", descricao: "Sensor de alta precisão." },
  { id: "3", nome: "Headset Gamer", descricao: "Áudio imersivo 7.1." },
];

export default function ProdutoDetalhe() {
  const { id } = useParams();
  const navigate = useNavigate();
  const produto = produtos.find((p) => p.id === id);

  if (!produto) {
    return <h1>Produto não encontrado</h1>;
  }

  return (
    <div>
      <h1>{produto.nome}</h1>
      <p>{produto.descricao}</p>
      <button onClick={() => navigate("/produtos")}>Voltar para a lista</button>
    </div>
  );
}
```

Explicando bloco a bloco

- `const { id } = useParams();` — lê o parâmetro da rota atual, exatamente como na seção anterior.
- `produtos.find((p) => p.id === id)` — **procura**, dentro do array, o produto cujo `id` bate com o parâmetro recebido da URL. `find()` devolve o primeiro item que bater com a condição, ou `undefined` se nenhum bater.
- `if (!produto) { return <h1>Produto não encontrado</h1>; }` — tratamento de segurança: se alguém digitar um endereço com um `id` que não existe no array (ex.: `/produtos/99`), mostra uma mensagem em vez de a página quebrar tentando ler `produto.nome` de algo que não existe.
- O restante do JSX só aparece **depois** desse "portão de segurança" — nesse ponto, o React já sabe que `produto` existe de verdade.

4. Navegação Programática

Mudando de página a partir do código, sem clicar num link.

Repare no botão "Voltar para a lista" do exemplo anterior. Ele não é um `Link` — é um botão comum, que muda de rota **a partir de código**, usando o hook `useNavigate`:

```
import { useNavigate } from "react-router-dom";

const navigate = useNavigate();

<button onClick={() => navigate("/produtos")}>Voltar para a lista</button>
```

`navigate("/produtos")` manda o usuário para a rota `/produtos`, por código — o mesmo efeito de clicar num `Link`, mas disparado dentro de uma função (por exemplo, ao clicar num botão, depois de validar um formulário, ou após concluir uma ação).

Link × useNavigate

Use `Link` quando a navegação é o próprio conteúdo clicável (um menu, o nome de um produto na lista). Use `useNavigate` quando a navegação é uma **consequência** de uma ação (clicar num botão "Voltar", enviar um formulário, finalizar uma compra).

5. Página 404 — A Rota Coringa

Tratando endereços que não existem em nenhuma rota.

E se o usuário digitar um endereço que não existe, tipo `/qualquercoisa`? Sem tratamento, a tela fica em branco. A solução é uma rota **coringa**, usando `*` como caminho — ela bate com **qualquer** endereço que nenhuma outra rota tenha capturado:

`src/pages/NotFound.jsx`

```
export default function NotFound() {  
  return <h1>404 - Página não encontrada</h1>;  
}
```

Juntando tudo no `App.jsx`

`src/App.jsx`

```

import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
import Home from "../pages/Home";
import Sobre from "../pages/Sobre";
import Contato from "../pages/Contato";
import Ajuda from "../pages/Ajuda";
import Produtos from "../pages/Produtos";
import ProdutoDetalhe from "../pages/ProdutoDetalhe";
import NotFound from "../pages/NotFound";

export default function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Início</Link>
        <Link to="/sobre">Sobre</Link>
        <Link to="/contato">Contato</Link>
        <Link to="/ajuda">Ajuda</Link>
        <Link to="/produtos">Produtos</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/sobre" element={<Sobre />} />
        <Route path="/contato" element={<Contato />} />
        <Route path="/ajuda" element={<Ajuda />} />
        <Route path="/produtos" element={<Produtos />} />
        <Route path="/produtos/:id" element={<ProdutoDetalhe />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}

```

O que mudou em relação à Aula 1: um novo link no menu (`/produtos`) e três novas rotas (`/produtos` , `/produtos/:id` e a coringa `*`).

Atenção à ordem

A rota `path="*"` deve ser sempre a **última** dentro do `Routes` . O React Router lê de cima para baixo e usa a primeira que bater — se o coringa vier antes das outras, ele "rouba" todas as rotas seguintes, e nenhuma outra página nunca mais aparece.

Resumo geral desta aula

Peça	Para que serve
<code>:id</code> no <code>path</code>	Marca uma parte do caminho como dinâmica (ex.: <code>/produtos/:id</code>).
<code>useParams()</code>	Lê o valor do parâmetro da rota atual (sempre como <code>string</code>).
<code>array.find(...)</code>	Busca, dentro de um <code>array</code> , o item cujo campo bate com o parâmetro recebido.
<code>if (!item) { ... }</code>	Trata o caso de o parâmetro não corresponder a nenhum item do <code>array</code> .
<code>useNavigate()</code>	Muda de rota a partir do código, sem precisar de um <code>Link</code> .
<code>path="*"</code>	Rota coringa — captura qualquer endereço não previsto (página 404). Deve ficar por último.

Desenvolvimento Web — Guia de Estudos • Rotas Parte 2: Parâmetros e Navegação Programática • Projeto de referência: Routes (Home, Sobre, Contato, Ajuda, Produtos).